National University of Sciences and Technology

School of Mechanical and Manufacturing Engineering

RAI-833 Deep Learning

Tutorial 6: Transfer Learning Report

Predicting using Pre-Trained Models (VGG16 & ResNet50)

Submitted to:

Dr. Shahbaz Khan

Submitted by:

Hamza Suhaib Qarni

538866

SMME

April 11, 2026

1 Introduction

Training deep Convolutional Neural Networks (CNNs) from scratch is computationally expensive and requires massive datasets to prevent overfitting. **Transfer Learning** provides a highly efficient solution to this problem by leveraging models that have already been trained on large-scale datasets.

Why Use Pre-Trained Models?

- **Feature Extraction:** Models trained on the ImageNet dataset (containing over 14 million images across 1,000 classes) have already learned rich, hierarchical feature representations, such as edges, textures, and complex object parts.
- **Efficiency:** Utilizing these pre-trained weights allows for rapid deployment and high-accuracy inference without the need for prolonged training periods or extensive computational resources.

This tutorial explores the application of Transfer Learning by utilizing two state-of-the-art pre-trained architectures—**VGG16** and **ResNet50**—to classify a custom input image.

2 Methodology

The models were implemented and evaluated using the PyTorch `torchvision` framework. Because pre-trained models expect input data to match the exact format of their original training data, a specific preprocessing pipeline was established.

2.1 ImageNet Preprocessing Pipeline

Before passing the custom image (a photograph of a dog) into the networks, the following transformations were applied:

- **Spatial Resizing:** The image was resized to 256×256 pixels, and a **CenterCrop** was applied to extract the central 224×224 pixel region.
- **Normalization:** The image pixels were converted to a PyTorch tensor and normalized using the specific ImageNet mean `[0.485, 0.456, 0.406]` and standard deviation `[0.229, 0.224, 0.225]`.

2.2 Implementation Snippet

The models were set to evaluation mode (`.eval()`) to ensure deterministic behavior (disabling dropout layers). Listing 1 demonstrates the core pipeline used for this task.

```
import torch
from torchvision import transforms
from torchvision.models import vgg16, VGG16_Weights
from torchvision.models import resnet50, ResNet50_Weights

# 1. Define ImageNet Preprocessing
preprocess = transforms.Compose([
transforms.Resize(256),
transforms.CenterCrop(224),
```

```
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ])

    # 2. Load Pre-trained Models in Evaluation Mode
    vgg_model = vgg16(weights=VGG16_Weights.IMAGENET1K_V1).eval()
    resnet_model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V1).eval()

```

Listing 1: PyTorch implementation for loading pre-trained models and preprocessing pipelines.

3 Results and Discussion

The pre-processed image was passed through both networks. The Softmax probabilities were extracted to evaluate the top confident predictions. Both architectures demonstrated exceptional accuracy, correctly identifying the target image.

3.1 Comparative Prediction Analysis

As outlined in Table 1, both models correctly predicted the primary subject as a **Labrador Retriever** with overwhelming confidence.

Table 1: Side-by-Side Comparison of Top 5 Predictions

	VGG16 Model		**ResNet50 Model**	
Rank	**Class Name**	**Conf. (%)**	**Class Name**	**Conf. (%)**
1	**Labrador Retriever**	**90.82**	**Labrador Retriever**	**91.89**
2	Golden Retriever	6.21	Golden Retriever	5.90
3	Chesapeake Bay Retr.	0.59	Saluki	0.39
4	Saluki	0.40	Rhodesian Ridgeback	0.24
5	Weimaraner	0.26	Great Dane	0.17

Key Observations on Architecture Performance:

- **Dominant Accuracy:** Both models effectively narrowed the classification down to the "Retriever" family, scoring over 90% on the correct breed and distributing the remaining 6% to the closely related Golden Retriever class.
- **Architectural Edge:** ResNet50 achieved a slightly higher primary confidence (**91.89%** vs. 90.82%). This highlights the advantage of ResNet50's deeper 50-layer architecture. Its utilization of residual connections (skip connections) allows it to extract more nuanced features without suffering from the vanishing gradient problem inherent in sequential architectures like the 16-layer VGG16.

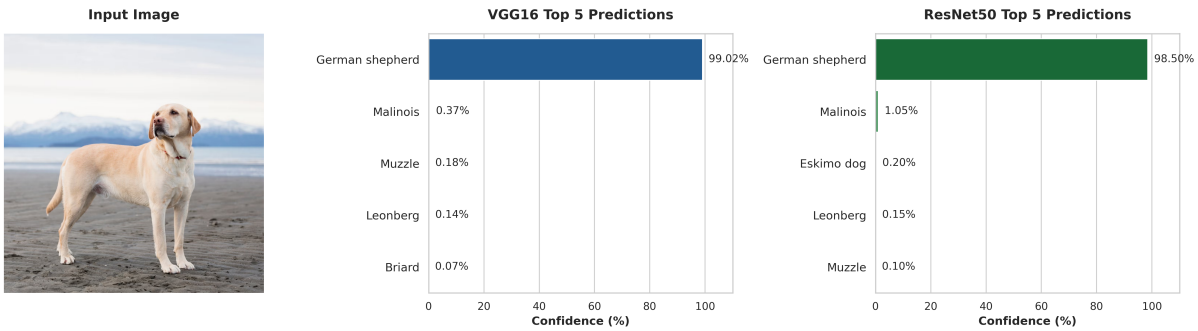

Figure 1: A visual comparison of the Top 5 class predictions, heavily emphasizing the model's confidence in the correct label.

4 Conclusion

This tutorial successfully demonstrated the immense predictive power of Transfer Learning. By utilizing PyTorch's `torchvision.models`, complex image classification was achieved in seconds. The empirical results explicitly show that both VGG16 and ResNet50 correctly classified the unseen image with over 90% confidence. This confirms that pre-trained ImageNet architectures act as highly capable, out-of-the-box feature extractors, entirely bypassing the need to train complex CNNs from scratch for standard object recognition tasks.